

Python_Code – student_search.py

Implementation of the three search algorithms, tests and experiment driver (Assessment-1, Design and Analysis of Algorithms)

```
1  """
2  Student Record Retrieval - three search strategies, experiments for Assignment 1 (DAA).
3  A. Linear search      (iterative, no preprocessing)
4  B. Binary search     (recursive, preprocessing = sort)
5  C. Hash table search (iterative, preprocessing = build table, separate chaining)
6  Run: python student_search.py      (writes results.json)
7  """
8  import random, time, json, math, sys
9  sys.setrecursionlimit(10000)
10 SEED = 42
11
12 # ----- Algorithm A: linear search (iterative) -----
13 def linear_search(a, key):
14     for i, x in enumerate(a):
15         if x == key:
16             return i
17     return -1
18
19 def linear_search_count(a, key):
20     c = 0
21     for i, x in enumerate(a):
22         c += 1
23         if x == key:
24             return i, c
25     return -1, c
26
27 # ----- Algorithm B: binary search (recursive) -----
28 def binary_search_rec(a, key, lo, hi):
29     if lo > hi:
30         return -1
31     mid = (lo + hi) // 2
32     if a[mid] == key:
33         return mid
34     if key < a[mid]:
35         return binary_search_rec(a, key, lo, mid - 1)
36     return binary_search_rec(a, key, mid + 1, hi)
37
38 def binary_search_count(a, key, lo, hi, c=0):
39     if lo > hi:
40         return -1, c
41     mid = (lo + hi) // 2
42     c += 1
43     if a[mid] == key:
44         return mid, c
45     if key < a[mid]:
46         return binary_search_count(a, key, lo, mid - 1, c)
47     return binary_search_count(a, key, mid + 1, hi, c)
48
49 def preprocess_sort(keys):
50     return sorted(keys)
51
52 # ----- Algorithm C: hash table, separate chaining -----
53 def next_prime(x):
54     def is_p(v):
55         if v < 2: return False
56         i = 2
57         while i * i <= v:
58             if v % i == 0: return False
59             i += 1
60         return True
61     while not is_p(x): x += 1
62     return x
```

```

63
64 def build_hash(keys, load=1.3):
65     m = next_prime(int(len(keys) * load))
66     table = [[] for _ in range(m)]
67     for k in keys:
68         table[k % m].append(k)          # h(k) = k mod m
69     return table
70
71 def hash_search(table, key):
72     chain = table[key % len(table)]
73     for i, x in enumerate(chain):
74         if x == key:
75             return i
76     return -1
77
78 def hash_search_count(table, key):
79     chain = table[key % len(table)]
80     c = 0
81     for i, x in enumerate(chain):
82         c += 1
83         if x == key:
84             return i, c
85     return -1, c
86
87 # ----- helpers -----
88 def make_queries(keys, n_hit=100, n_miss=100, rng=None):
89     hits = rng.sample(keys, n_hit)
90     s = set(keys); misses = []
91     while len(misses) < n_miss:
92         q = rng.randrange(10**8, 10**9)
93         if q not in s: misses.append(q)
94     return hits, misses
95
96 def best_time(fn, reps):
97     best = float("inf")
98     for _ in range(reps):
99         t0 = time.perf_counter(); fn(); t = time.perf_counter() - t0
100         best = min(best, t)
101     return best
102
103 def correctness_tests():
104     rng = random.Random(1)
105     # edge cases
106     for keys in ([], [5], [3, 9], [7, 1, 9, 4, 12, 30, 2]):
107         s = sorted(keys); h = build_hash(keys) if keys else [[]]
108         probes = keys + [0, 100, 6, 10, 31]
109         for q in probes:
110             exp = q in keys
111             assert (linear_search(keys, q) != -1) == exp
112             assert (binary_search_rec(s, q, 0, len(s) - 1) != -1) == exp
113             assert (hash_search(h, q) != -1) == exp
114     # random
115     keys = rng.sample(range(10**8, 10**9), 5000)
116     s = sorted(keys); h = build_hash(keys); S = set(keys)
117     hits, misses = make_queries(keys, 500, 500, rng)
118     for q in hits + misses:
119         exp = q in S
120         i = linear_search(keys, q); j = binary_search_rec(s, q, 0, len(s) - 1); k = hash_search(h, q)
121         assert (i != -1) == exp and (j != -1) == exp and (k != -1) == exp
122         if exp: assert keys[i] == q and s[j] == q
123     return True
124

```

```

125 def main():
126     assert correctness_tests()
127     rng = random.Random(SEED)
128     master = rng.sample(range(10**8, 10**9), 1_000_000) # unique 9-digit registration numbers
129     sizes = [1000, 10_000, 100_000, 1_000_000]
130     res = {"sizes": sizes, "pre": {}, "avg": {}, "cases": {}, "meta": {}}
131     for n in sizes:
132         keys = master[:n]
133         hits, misses = make_queries(keys, 100, 100, random.Random(n))
134         queries = hits + misses
135         # ---- preprocessing
136         t_sort = best_time(lambda: preprocess_sort(keys), 3 if n >= 100_000 else 20)
137         t_hash = best_time(lambda: build_hash(keys), 3 if n >= 100_000 else 20)
138         s = preprocess_sort(keys); h = build_hash(keys); hi = n - 1
139         res["pre"][n] = {"sort_s": t_sort, "hash_s": t_hash,
140                         "buckets": len(h), "max_chain": max(len(c) for c in h),
141                         "load": n / len(h)}
142         # ---- correctness on this dataset
143         S = set(keys)
144         for q in queries:
145             e = q in S
146             assert (linear_search(keys, q) != -1) == e
147             assert (binary_search_rec(s, q, 0, hi) != -1) == e
148             assert (hash_search(h, q) != -1) == e
149         # ---- operation counts (instrumented versions)
150         def counts(fn, qs): return sum(fn(q)[1] for q in qs) / len(qs)
151         cl = lambda q: linear_search_count(keys, q)
152         cb = lambda q: binary_search_count(s, q, 0, hi)
153         ch = lambda q: hash_search_count(h, q)
154         cnt = {"linear": {"hit": counts(cl, hits), "miss": counts(cl, misses)},
155               "binary": {"hit": counts(cb, hits), "miss": counts(cb, misses)},
156               "hash": {"hit": counts(ch, hits), "miss": counts(ch, misses)}}
157         # ---- timing (uninstrumented versions), min over repetitions, per search
158         reps_slow = 3 if n >= 100_000 else 10
159         reps_fast = 50
160         def per_search(fn, qs, reps):
161             return best_time(lambda: [fn(q) for q in qs], reps) / len(qs)
162         tm = {}
163         for name, fn, reps in (("linear", lambda q: linear_search(keys, q), reps_slow),
164                               ("binary", lambda q: binary_search_rec(s, q, 0, hi), reps_fast),
165                               ("hash", lambda q: hash_search(h, q), reps_fast)):
166             tm[name] = {"hit": per_search(fn, hits, reps), "miss": per_search(fn, misses, reps)}
167         res["avg"][n] = {"count": cnt, "time": tm}
168         # ---- best / worst case probes
169         mid_key = s[(n - 1) // 2]
170         worst_b = s[0] # leftmost element: longest probe path
171         # hash best case: first element of a non-empty chain; hash worst: adversarial keys
172         m = len(h)
173         adv_keys = [i * m for i in range(1, n + 1)] # all hash to bucket 0
174         adv = build_hash(adv_keys)
175         def single(fn, reps=5): return best_time(fn, reps)
176         hb_key = h[next(i for i, c in enumerate(h) if c)][0] # a key sitting first in its chain
177         cases = {
178             "linear": {"best": (linear_search_count(keys, keys[0])[1], single(lambda: linear_search(keys, keys[0]),
179             "worst_hit": (linear_search_count(keys, keys[-1])[1], single(lambda: linear_search(keys, keys[-1]),
180             "miss": (linear_search_count(keys, misses[0])[1], single(lambda: linear_search(keys, misses[0]),
181             "binary": {"best": (binary_search_count(s, mid_key, 0, hi)[1], single(lambda: binary_search_rec(s, mid_key, 0, hi),
182             "worst_hit": (binary_search_count(s, worst_b, 0, hi)[1], single(lambda: binary_search_rec(s, worst_b, 0, hi),
183             "miss": (binary_search_count(s, misses[0], 0, hi)[1], single(lambda: binary_search_rec(s, misses[0], 0, hi),
184             "hash": {"best": (hash_search_count(h, hb_key)[1], single(lambda: hash_search(h, hb_key), 50)),
185             "worst_hit": (hash_search_count(adv, adv_keys[-1])[1], single(lambda: hash_search(adv, adv_keys[-1]),

```

```

187         "miss": (hash_search_count(h, misses[0])[1], single(lambda: hash_search(h, misses[0]),
188     }
189     res["cases"][n] = cases
190     print("done", n, flush=True)
191
192     # ----- Task 12: 50,000 requests/hour on 1,000,000 records -----
193     keys = master; n = len(keys)
194     s = sorted(keys); h = build_hash(keys); hi = n - 1
195     rng2 = random.Random(2026)
196     hits, misses = make_queries(keys, 25_000, 25_000, rng2) # 50,000 requests, 50% hits / 50% misses
197     reqs = hits + misses; rng2.shuffle(reqs)
198     t0 = time.perf_counter(); [binary_search_rec(s, q, 0, hi) for q in reqs]; tb = time.perf_counter() - t0
199     t0 = time.perf_counter(); [hash_search(h, q) for q in reqs]; th = time.perf_counter() - t0
200     sample = reqs[:300]
201     t0 = time.perf_counter(); [linear_search(keys, q) for q in sample]; tl_s = time.perf_counter() - t0
202     res["task12"] = {"requests": len(reqs), "binary_total_s": tb, "hash_total_s": th,
203         "linear_sample_n": len(sample), "linear_sample_s": tl_s,
204         "linear_est_total_s": tl_s / len(sample) * len(reqs)}
205     # daily rebuild after the once-a-day insertion of (say) 2,000 new records
206     new = rng2.sample(range(10**9, 2 * 10**9), 2000)
207     t0 = time.perf_counter(); merged = sorted(keys + new); res["task12"]["resort_s"] = time.perf_counter() - t0
208     t0 = time.perf_counter(); build_hash(keys + new); res["task12"]["rehash_s"] = time.perf_counter() - t0
209     json.dump(res, open("results.json", "w"), indent=1)
210     print("saved")
211
212 if __name__ == "__main__":
213     main()
214

```